Leme

Arquitetura do banco de dados

Modelo de dados, formato binário do registro, segmentos de log, índice em memória, recuperação, compactação, versionamento com relógio híbrido, TTL e lápides, anel de hash, replicação com quórum, protocolo, modelagem de chaves e backup lógico.

Versão 1.1.0 · Outubro de 2026

Copyright (©) 2026 Erik Pimenta

Sumário

01 Visão geral 4

1.1 Objetivos e escolhas 4
1.2 O que o banco não é 4
1.3 Camadas 4

02 Modelo de dados 5

2.1 O item 5
2.2 Regras 5
2.3 Ordem das chaves 5

03 Formato do registro 6

3.1 Layout 6
3.2 Exemplo 6
3.3 Por que CRC32C 6

04 Segmentos de log 7

4.1 Arquivos 7
4.2 Rotação 7
4.3 Sincronização com o disco 7

05 Índice em memória 8

5.1 Entrada do índice 8
5.2 Custo de memória 8
5.3 Operações 8

06 Escrita, leitura e varredura 10

6.1 Escrita local 10
6.2 Leitura local 10
6.3 Varredura 10

07 Durabilidade e recuperação 11

7.1 Abertura do nó 11
7.2 Por que a ordem dos segmentos não importa 11
7.3 Registros descartados 11

08 Versionamento e conflitos 12

8.1 Relógio híbrido 12
8.2 A última escrita vence 12
8.3 Escrita condicional 12
8.4 Contadores, cópias e renomeações 13

Copyright (©) 2026 Erik Pimenta

09 TTL e lápides 14
9.1 Expiração 14
9.2 Lápides 14
9.3 Período de graça 14
10 Compactação 15
10.1 Algoritmo 15
10.2 Garantias 15
10.3 Quando compactar 15
11 Distribuição pelo anel 16
11.1 Hash consistente 16
11.2 Réplicas de uma chave 16
11.3 Mudanças de membros 16
12 Replicação e consistência 17
12.1 N, R e W 17
12.2 Escrita no cluster 17
12.3 Leitura no cluster 17
12.4 Membros e falhas 18
12.5 Redistribuição 18
13 Varredura distribuída 19
13.1 O problema 19
13.2 O algoritmo 19
14 Protocolo binário 20
14.1 Quadro 20
14.2 Operações 20
14.3 Códigos de erro 21
15 Modelando as chaves 22
15.1 Convenções 22
15.2 Esquema de exemplo: loja 22
15.3 Padrões 22
15.4 O que evitar 23
16 Escalabilidade e operação 24
16.1 Capacidade 24
16.2 Crescendo 24
16.3 Backup e restauração 24
16.4 Backup lógico 24
16.5 Métricas do banco 25
16.6 Diagnóstico 25

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 01

Visão geral

O banco do Leme é um armazenamento chave-valor distribuído. Cada nó guarda a sua parte das chaves num log só de acréscimo com um índice em memória, no estilo do Bitcask, e os nós se combinam num anel de hash consistente com replicação por quórum, no estilo do Dynamo. Este documento descreve como os dados são organizados, gravados, lidos, versionados, distribuídos e recuperados, e como modelar as suas chaves para tirar proveito disso.

1.1 Objetivos e escolhas

OBJETIVO	ESCOLHA
Escritas rápidas e previsíveis	Cada escrita é um acréscimo no fim de um arquivo; nada é reescrito no lugar
Leitura com um acesso a disco	O índice em memória aponta direto para a posição do registro
Sobreviver a quedas	Cada registro leva CRC32C; na abertura, a cauda inválida é truncada
Tolerar a queda de nós	Cada chave tem N réplicas; leituras e escritas exigem só a maioria
Crescer sem parar	Hash consistente com nós virtuais move só a fração necessária das chaves
Resolver conflitos sem coordenação	Versões de relógio híbrido; vence a maior
Simplicidade de operação	Um binário, uma pasta por nó, sem dependências externas

1.2 O que o banco não é

O Leme não tem transações entre chaves, consultas por valor, índices secundários nem ordenação global por outro critério além do nome da chave. Esses recursos se constroem por cima, com a modelagem das chaves descrita no capítulo 15: prefixos para agrupar, chaves de índice mantidas pela aplicação e escrita condicional para controle de concorrência, que é o que os comandos `incr`, `decr` e `rename` da CLI fazem.

1.3 Camadas

CLI / console	`leme get · put · scan · incr · export` e a API REST do console, falando o protocolo binário sobre TCP (TLS opcional)
↓	
Coordenador	anel → N réplicas · versão do relógio híbrido · quórum R/W · dicas · reparo de leitura
↓	
Réplica (cada nó)	índice em memória `chave → (segmento, posição, tamanho, versão, expira, lápide)` —▶ segmentos de log `00000001.log` `00000002.log` …

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 02

Modelo de dados

Tudo no banco é um item: uma chave, um valor e três metadados. Este capítulo define cada campo e as regras que valem para eles.

2.1 O item

CAMPO	TIPO	DESCRIÇÃO
`chave`	bytes, 1 a 65.536	Identificador único do item no cluster. Comparada byte a byte
`valor`	bytes, 0 a 33.554.432	Conteúdo opaco: texto, JSON, binário. O banco nunca interpreta
`versao`	inteiro de 64 bits	Relógio híbrido do coordenador no momento da escrita (capítulo 8)
`expira`	inteiro de 64 bits	Instante de expiração em milissegundos Unix; 0 é sem expiração
`lapide`	booleano	Marca de exclusão; um item com lápide não existe para leituras

2.2 Regras

REGRA	EFEITO
Chave vazia	Recusada com `requisicao_invalida`
Chave acima de 64 KiB ou valor acima de 32 MiB	Recusados antes de chegar ao disco
Duas escritas na mesma chave	Vence a versão maior, em todas as réplicas
Item expirado	Some das leituras e varreduras imediatamente; sai do disco na compactação
Item apagado	Vira lápide; sai do disco na compactação depois do período de graça
Valor vazio	Válido: a chave existe, com zero bytes

2.3 Ordem das chaves

As chaves são ordenadas lexicograficamente pelos bytes. Isso define a ordem do `scan`, a paginação por `--depois` e a ordem em que `export` grava o arquivo. Para que números ordenem corretamente, use zeros à esquerda: `pedido:000042` vem antes de `pedido:000100`, mas `pedido:42` vem depois de `pedido:100`.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 03

Formato do registro

Cada escrita vira um registro binário acrescentado ao segmento ativo. O formato é o mesmo para valores e lápides.

3.1 Layout

DESLOCAMENTO	TAMANHO	CAMPO	CODIFICAÇÃO
0	4 bytes	`crc`	CRC32C (Castagnoli) de todos os bytes a partir do deslocamento 4, big-endian
4	1 byte	`tipo`	1 valor, 2 lápide
5	8 bytes	`versao`	inteiro sem sinal, big-endian
13	8 bytes	`expira`	milissegundos Unix, big-endian; 0 sem expiração
21	4 bytes	`tamanho_chave`	inteiro sem sinal, big-endian, de 1 a 65.536
25	4 bytes	`tamanho_valor`	inteiro sem sinal, big-endian, até 33.554.432
29	`tamanho_chave`	`chave`	bytes
29 + chave	`tamanho_valor`	`valor`	bytes; vazio nas lápides

O cabeçalho tem 29 bytes. O registro inteiro ocupa 29 + chave + valor bytes.

3.2 Exemplo

O registro de `put usuario:1 Ana`, com versão `0x01A1B2C3D4E50000` e sem expiração, tem 41 bytes:

CRC	TIPO	VERSAO	EXPIRA	TAM_CHAVE
F4 F6 4B 27	01	01 A1 B2 C3 D4 E5 00 00	00 00 00 00 00 00 00 00	00 00 00 09

TAM_VALOR	CHAVE	VALOR
00 00 00 03	75 73 75 61 72 69 6F 3A 31	41 6E 61

3.3 Por que CRC32C

O CRC32C detecta qualquer erro de 1 a 3 bits e todas as rajadas de até 32 bits, que são as corrupções típicas de setores e de escritas interrompidas. Ele cobre o tipo, a versão, a expiração, os tamanhos, a chave e o valor. Um tamanho absurdo no cabeçalho (tipo desconhecido, chave vazia, chave ou valor acima do limite) é rejeitado antes mesmo de ler o corpo, o que impede que um cabeçalho corrompido provoque leituras gigantes.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 04

Segmentos de log

Os registros ficam em arquivos só de acréscimo, os segmentos. Só um deles recebe escritas; os outros são imutáveis.

4.1 Arquivos

Os registros ficam em segmentos numerados na pasta do nó:

```
dados/
  no-1/
    00000001.log     fechado
    00000002.log     fechado
    00000007.log     gerado pela compactação
    00000008.log     ativo: recebe as escritas
```

O nome é o número do segmento com oito dígitos. Números novos são sempre maiores que os existentes. Só o segmento ativo recebe escritas; os outros são imutáveis e só são lidos ou removidos pela compactação.

4.2 Rotação

Quando uma escrita faria o segmento ativo passar de `--segmento-mb` (64 MiB por padrão), o nó sincroniza o ativo com o disco, fecha-o e abre um novo. Um registro nunca é dividido entre dois segmentos.

4.3 Sincronização com o disco

MODO	QUANDO O FSYNC ACONTECE	RISCO NUMA QUEDA DE ENERGIA
`sempre`	Depois de cada escrita, antes de responder	Nenhuma escrita confirmada é perdida
`intervalo` (padrão)	A cada segundo, se houve escrita; e ao rotacionar, compactar e fechar	Até 1 segundo de escritas daquele nó; as outras réplicas cobrem
`nunca`	Só ao rotacionar, compactar e fechar	O que o sistema operacional ainda não gravou

Com três réplicas e quórum, uma escrita confirmada está em pelo menos dois nós. Perder o último segundo de um nó não perde a escrita: o reparo de leitura e as dicas a devolvem.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 05

Índice em memória

Cada nó mantém na memória um mapa da chave para a posição do registro mais novo, e é ele que faz cada leitura custar um único acesso ao disco.

5.1 Entrada do índice

CAMPO	TIPO	DESCRIÇÃO
`seg`	inteiro de 32 bits	Número do segmento
`pos`	inteiro de 64 bits	Deslocamento do registro no segmento
`tam`	inteiro de 64 bits	Tamanho do registro em bytes
`versao`	inteiro de 64 bits	Versão do registro
`expira`	inteiro de 64 bits	Expiração, para filtrar sem ler o disco
`lapide`	booleano	Se o registro é uma lápide

5.2 Custo de memória

Cada entrada ocupa perto de 100 bytes mais o tamanho da chave. Estimativas por nó:

CHAVES NO NÓ	CHAVE MÉDIA	MEMÓRIA DO ÍNDICE
100 mil	20 bytes	~12 MiB
1 milhão	30 bytes	~130 MiB
10 milhões	30 bytes	~1,3 GiB
10 milhões	100 bytes	~2 GiB

Com três réplicas, cada nó guarda cerca de 3 × total / número de nós chaves. Para mais chaves, adicione nós.

5.3 Operações

OPERAÇÃO	ÍNDICE	DISCO
Gravar	Escrita exclusiva, O(1)	Um acréscimo
Ler	Leitura compartilhada, O(1)	Uma leitura posicionada
Varrer por prefixo	Leitura compartilhada, O(n) para filtrar mais O(k log k) para ordenar	Uma leitura por item devolvido, ou nenhuma em `keys`
Estatísticas	Leitura compartilhada, O(n)	Nenhuma

Copyright (©) 2026 Erik Pimenta

Leituras e varreduras acontecem em paralelo; só gravações e a troca final da compactação seguram o índice com exclusividade, por microssegundos.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 06

Escrita, leitura e varredura

O caminho de uma operação dentro de um nó, do pedido ao disco.

6.1 Escrita local

01 Valida chave e valor.

02 Com o índice travado, compara a versão nova com a atual da chave. Se não for maior, a escrita é ignorada: a réplica já tem algo igual ou mais novo, e isso conta como confirmação.

03 Codifica o registro, calcula o CRC e acrescenta ao segmento ativo, rotacionando se preciso.

04 Sincroniza se o modo for `sempre`.

05 Aponta o índice para a nova posição.

6.2 Leitura local

01 Busca a chave no índice.

02 Lê o registro na posição indicada e confere o CRC.

03 Para o cliente, devolve “não encontrada” se for lápide ou se já expirou. Para a replicação, devolve o registro como está, com versão e lápide, porque o coordenador precisa comparar versões.

6.3 Varredura

A varredura filtra o índice pelo prefixo e pela chave inicial (`--depois`), ordena e lê os valores só das chaves devolvidas. No cluster, o coordenador pede a cada nó vivo as suas chaves daquele trecho, incluindo lápides, junta pela versão mais nova, descarta lápides e expiradas e devolve a página. Detalhes no capítulo 13. `count`, `keys`, `export` e `watch --prefixo` são construídos sobre a mesma varredura paginada.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO **07**

Durabilidade e recuperação

Como um nó reconstrói o índice ao abrir e o que acontece com uma escrita interrompida por uma queda.

7.1 Abertura do nó

01 Lista os segmentos da pasta e ordena pelo número.

02 Lê cada segmento do início ao fim, registro a registro, conferindo cabeçalho e CRC.

03 Para cada registro válido, atualiza o índice se a versão for maior que a já vista para a chave.

04 No primeiro registro inválido de um segmento, para a leitura daquele segmento. Se for o último segmento, trunca o arquivo naquela posição, removendo a escrita interrompida.

05 Continua no segmento ativo se ele ainda tiver espaço; senão, abre um novo.

7.2 Por que a ordem dos segmentos não importa

O índice é reconstruído pela versão de cada registro, não pela posição. Mesmo que a compactação gere um segmento com número maior contendo registros antigos, a versão mais nova continua vencendo. A ordem só serve para saber qual é o último segmento, o único que pode ter uma escrita incompleta.

7.3 Registros descartados

`leme node stats` mostra quantos registros foram descartados na última abertura. Um descartado no fim do último segmento é normal depois de uma queda de energia. Descartados no meio de segmentos fechados indicam problema de disco: troque o disco e deixe as réplicas repovoarem o nó.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 08

Versionamento e conflitos

Toda escrita leva uma versão de 64 bits, e é ela que decide conflitos entre réplicas e sustenta as escritas condicionais.

8.1 Relógio híbrido

A versão é gerada pelo nó que coordena a escrita:

BITS	CONTEÚDO
63 a 16 (48 bits)	Milissegundos desde 1970, suficientes até o ano 10889
15 a 0 (16 bits)	Contador, para mais de uma escrita no mesmo milissegundo

A regra: `versao = max(agora_ms << 16, ultima + 1)`. Ao receber uma réplica com versão maior que a sua, o nó avança o relógio até ela. Assim as versões geradas por um nó nunca andam para trás, mesmo se o relógio do sistema voltar, e acompanham o tempo real quando os relógios estão sincronizados.

8.2 A última escrita vence

Entre duas versões de uma chave, vence a maior. No raro caso de versões iguais (dois coordenadores no mesmo milissegundo e contador), o coordenador prefere a lápide e, entre valores, o de maior ordem de bytes; cada réplica mantém o primeiro que recebeu com aquela versão. Mantenha os relógios sincronizados por NTP: com relógios divergentes, a escrita de um nó adiantado pode vencer uma escrita posterior de um nó atrasado. O `leme doctor` mostra a versão do `lemed` de cada nó, um bom momento para conferir também o NTP.

8.3 Escrita condicional

`--se-versao` e `--se-ausente` implementam controle de concorrência otimista:

01 O coordenador lê a chave com quórum.

02 Compara a versão atual com a esperada, ou verifica que a chave não existe.

03 Se a condição falhar, responde `conflito_de_versao` (código 6) sem gravar.

04 Se passar, grava com uma versão nova.

A verificação e a gravação não são atômicas entre coordenadores diferentes: duas escritas condicionais simultâneas por coordenadores distintos podem ambas passar, e vence a de versão maior. Para contadores e estoques de alta disputa, faça as escritas da mesma chave pelo mesmo coordenador ou serialize na aplicação.

Copyright (©) 2026 Erik Pimenta

8.4 Contadores, cópias e renomeações

Os comandos da 1.1.0 que mudam uma chave a partir do valor atual usam a escrita condicional:

COMANDO	COMO FUNCIONA
`incr·decr`	Lê valor e versão, soma o delta e grava com `--se-versao`; num conflito, lê de novo e repete com espera crescente. Chave ausente começa em 0 e é criada com `--se-ausente`
`copy`	Lê a origem e grava o destino com o mesmo valor e o TTL restante; sem `--substituir`, grava com `--se-ausente`
`rename`	Faz o `copy` e depois apaga a origem com `--se-versao` igual à versão lida, para não apagar uma escrita feita no meio do caminho

As mesmas ressalvas de atomicidade da seção anterior valem para eles: são seguros contra escritas concorrentes que passam pelo mesmo coordenador e convergem pela maior versão nos demais casos.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 09

TTL e lápides

Chaves podem expirar sozinhas, e apagar uma chave deixa uma marca versionada no lugar.

9.1 Expiração

`put --ttl 30m` grava `expira = agora + 30 minutos`, calculado pelo coordenador. A partir desse instante a chave some de leituras, `exists`, `scan` e `count` em todas as réplicas, sem precisar de nenhuma escrita. O registro continua no disco até a compactação. `expire` grava uma versão nova com o mesmo valor e outro prazo; `expire <chave> 0` remove a expiração. `export` grava o tempo restante (`ttl_ms`), e `import` recalcula o prazo a partir do momento da importação.

9.2 Lápides

Apagar grava um registro do tipo lápide com versão nova. A lápide é necessária: sem ela, uma réplica que estava fora do ar durante o `del` devolveria o valor antigo e o reparo de leitura espalharia de novo a chave apagada.

9.3 Período de graça

A compactação só remove lápides com mais de `--graca-lapide` (24 horas por padrão). Esse tempo precisa ser maior que o tempo máximo que um nó pode ficar fora do ar e voltar. Um nó que passou mais tempo fora deve voltar vazio (pasta de dados apagada), para não reintroduzir chaves apagadas.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 10

Compactação

As escritas só acrescentam registros; a compactação recupera o espaço de versões antigas, lápides e chaves expiradas.

10.1 Algoritmo

01 Fecha o segmento ativo e abre um novo, que continua recebendo escritas.

02 Separa as entradas do índice que apontam para segmentos fechados.

03 Para cada uma, em ordem de segmento e posição: descarta se for lápide mais velha que o período de graça ou valor expirado; senão, copia o registro para o arquivo `compactacao.tmp`.

04 Sincroniza o arquivo temporário e o renomeia para um segmento novo.

05 Com o índice travado por um instante, aponta para o segmento novo as entradas que não mudaram durante a cópia, remove as descartadas, fecha e apaga os segmentos antigos.

10.2 Garantias

GARANTIA	COMO
Leituras e escritas continuam	A cópia acontece sem trava; só a troca final trava o índice
Nenhuma escrita se perde	Escritas durante a compactação vão para o segmento ativo e vencem pela versão
Queda no meio é segura	Até o rename, os segmentos antigos estão intactos e o temporário é descartado
Uma compactação por vez	Uma segunda chamada simultânea recebe conflito

10.3 Quando compactar

A fragmentação de `node stats` é `1 - bytes vivos / bytes em disco`. Compacte quando passar de 50%, ou agende `leme node compact todos` fora do horário de pico. O `leme doctor` e a página Diagnóstico do console avisam quando algum nó passa desse ponto.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 11

Distribuição pelo anel

Hash consistente com nós virtuais decide onde cada chave mora e quanto se move quando o cluster muda.

11.1 Hash consistente

O espaço de chaves é um anel de 2^{64} posições. A posição de uma chave é o FNV-1a de 64 bits da chave, passado por uma função de mistura (a finalização do MurmurHash3) para espalhar chaves parecidas. Cada nó ocupa `vnodes` posições, calculadas da mesma forma a partir de `id#0`, `id#1` e assim por diante.

11.2 Réplicas de uma chave

A partir da posição da chave, segue-se o anel no sentido horário e escolhem-se os primeiros N nós distintos. O primeiro é a réplica primária. Todos os nós e clientes calculam o mesmo resultado, porque o anel depende só da lista de membros, do número de vnodes e da função de hash.

Desvio da fatia de cada nó em relação à divisão perfeita, medido com a função de hash do Leme:

MEMBROS	VNODES	PONTOS NO ANEL	DESVIO MÉDIO	MAIOR DESVIO
3	128	384	6,4%	8,2%
5	128	640	9,2%	11,7%
10	128	1.280	11,2%	23,4%
10	256	2.560	5,0%	8,0%

Com dez nós ou mais, use `--vnodes 256` em todos eles. A verificação de anel do `leme doctor` mede esse desvio no cluster real.

11.3 Mudanças de membros

Quando um nó entra, ele assume fatias de vários nós ao mesmo tempo. Com M nós, cerca de 1/(M+1) das chaves mudam de dono. Os testes conferem que, ao passar de 4 para 5 nós, entre 10% e 30% das chaves mudam de primária. Nós em estado `saiu` deixam o anel; nós `suspeitos` ou `mortos` continuam nele, para que uma queda temporária não mova dados.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 12

Replicação e consistência

Cada chave vive em N réplicas; leituras e escritas escolhem quantas precisam responder.

12.1 N, R e W

SÍMBOLO	SIGNIFICADO	PADRÃO
N	Réplicas de cada chave (`--replicas`)	3
W	Confirmações para uma escrita	maioria: 2
R	Respostas para uma leitura	maioria: 2

Com `R + W > N`, todo conjunto de leitura cruza todo conjunto de escrita, e a leitura enxerga a última escrita confirmada. A consistência `um` usa 1 e `todos` usa N.

12.2 Escrita no cluster

01 O cliente envia ao nó que considera primário; qualquer nó pode coordenar.

02 O coordenador calcula as N réplicas, gera a versão e envia o registro a todas em paralelo.

03 Responde ao atingir W confirmações; as outras seguem em segundo plano.

04 Réplicas marcadas como mortas ou que falharam recebem uma dica: o coordenador guarda o registro e o entrega quando o gossip mostrar a réplica viva de novo. Cada coordenador guarda até 100 mil dicas, uma por chave e destino, sempre a versão mais nova.

05 Se não atingir W, responde `quorum_nao_atingido`. As réplicas que confirmaram mantêm o valor, que pode aparecer em leituras futuras: trate o erro como “talvez gravado” e repita a operação.

12.3 Leitura no cluster

01 O coordenador pede o registro bruto às N réplicas.

02 Ao receber R respostas, escolhe a versão mais nova e responde.

03 Em segundo plano, espera as outras respostas e envia a versão mais nova a toda réplica que respondeu com uma versão antiga ou sem a chave: o reparo de leitura.

Copyright (©) 2026 Erik Pimenta

12.4 Membros e falhas

ESTADO	QUANDO	EFEITO
`vivo`	Respondeu ao último gossip	Participa de tudo
`suspeito`	Sem resposta há 3 intervalos	Continua sendo consultado
`morto`	Sem resposta há 10 intervalos	Não é consultado; recebe dicas
`saiu`	Executou `node leave`	Fora do anel

Cada nó troca a lista de membros com todos os outros a cada intervalo. Um membro com geração maior (reiniciado) substitui o anterior. Nós de clusters com nomes diferentes recusam a troca.

12.5 Redistribuição

Quando o anel muda, cada nó percorre as suas chaves e envia cada registro às réplicas que ele tem no anel novo. A escrita nas réplicas segue a regra da maior versão, então reenviar é seguro. Na saída de um nó, ele recalcula o anel sem si mesmo e envia todas as suas chaves às novas réplicas, enquanto os outros nós aprendem a saída pelo gossip e recalculam o anel também.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 13

Varredura distribuída

Como montar uma página ordenada de chaves quando cada nó só conhece as suas.

13.1 O problema

Cada nó só sabe as chaves que guarda. Uma página ordenada do cluster precisa juntar as respostas de todos, e um nó que devolveu menos chaves não pode deixar buracos.

13.2 O algoritmo

01 O coordenador pede a cada nó vivo as primeiras L chaves (incluindo lápides) depois de `--depois`, só das que ele é réplica no anel atual.

02 Junta as respostas por chave, ficando com a versão mais nova.

03 Calcula a fronteira: a menor "última chave" entre os nós que ainda tinham mais. Até a fronteira, a junção é completa; depois dela, algum nó pode não ter mandado tudo.

04 Descarta lápides e expiradas até a fronteira e acrescenta à página.

05 Se a página não encheu e algum nó tinha mais, repete a partir da fronteira, dobrando L a cada rodada (de 64 até 4.096), por até 64 rodadas.

Assim a paginação não perde chaves e atravessa trechos cheios de lápides sem devolver páginas vazias. Os testes gravam e apagam 300 chaves antes de 5 válidas e conferem que a primeira página traz as válidas. O `export` percorre todas as páginas desse jeito e lê o valor de cada chave com a consistência escolhida.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 14

Protocolo binário

Clientes e nós conversam por quadros binários multiplexados sobre TCP.

14.1 Quadro

DESLOCAMENTO	TAMANHO	CAMPO
0	4 bytes	Tamanho do restante, big-endian, até 64 MiB
4	1 byte	Versão do protocolo (1)
5	1 byte	Operação
6	4 bytes	Identificador da requisição
10	1 byte	Reservado
11	variável	Corpo

Uma conexão leva várias requisições ao mesmo tempo; as respostas voltam com o mesmo identificador, em qualquer ordem. No corpo, inteiros pequenos usam varint, versões usam 8 bytes big-endian e bytes levam o tamanho em varint à frente. O decodificador confere cada tamanho contra o que resta do corpo.

14.2 Operações

CÓDIGO	OPERAÇÃO	USO
1	PING	Saúde; responde sem token
2	AUTH	Envia o token
3 · 4 · 5	GET · PUT · DEL	Dados, coordenados
6	SCAN	Varredura distribuída
7 · 8	MEMBROS · STATS	Topologia e estatísticas, em JSON
9 · 10	COMPACTAR · SAIR	Administração do nó
11 · 12	TTL · EXPIRAR	Expiração
13 · 14 · 15	REPLICA_PUT · REPLICA_GET · SCAN_LOCAL	Internas, entre nós
16 · 17	GOSSIP · ENTRAR	Membros
126 · 127	RESPOSTA · ERRO	Respostas

Os comandos novos da CLI (`incr`, `decr`, `copy`, `rename`, `mget`, `mset`, `export`, `import`, `watch`, `top` e `doctor`) não exigem operações novas: são compostos no cliente a partir de `GET`, `PUT`, `DEL`, `SCAN`, `TTL`, `MEMBROS` e `STATS`.

Copyright (©) 2026 Erik Pimenta

14.3 Códigos de erro

CÓDIGO	NOME	CÓDIGO DE SAÍDA DA CLI
1	nao_encontrado	3
2	requisicao_invalida	2
3	nao_autorizado	7
4	quorum_nao_atingido	5
5	conflito_de_versao	6
6	indisponivel	4
7	erro_interno	1
8	tempo_esgotado	4

O código de saída 8 da CLI (diagnóstico com falhas) não vem do protocolo: é calculado pelo `leme doctor` a partir das verificações.

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 15

Modelando as chaves

O banco só conhece chaves e valores; a organização dos dados vem dos nomes das chaves.

15.1 Convenções

REGRA	EXEMPLO
Tipo primeiro, depois o identificador, separados por `:`	`usuario:000042`
Hierarquia da esquerda para a direita	`usuario:000042:pedido:000315`
Zeros à esquerda em números	`pedido:000315`, não `pedido:315`
Datas no formato ISO, do maior para o menor	`acesso:2026-10-23:usuario:000042`
Só caracteres seguros: letras, números, `:`, `-`, `_`, `.`	evita aspas no shell
Valores em JSON quando têm campos	`{"nome":"Ana","plano":"pro"}`

15.2 Esquema de exemplo: loja

CHAVE	VALOR	OBSERVAÇÃO
`usuario:000042`	`{"nome":"Ana","email":"ana@exemplo.com"}`	O registro principal
`usuario-email:ana@exemplo.com`	`000042`	Índice para buscar por e-mail, mantido pela aplicação
`usuario:000042:pedido:000315`	`""`	Relação: `scan usuario:000042:pedido:` lista os pedidos da pessoa
`pedido:000315`	`{"cliente":"000042","total":129.90}`	O pedido
`estoque:sku-1042`	`87`	Atualizado com `decr` ou `--se-versao`
`sessao:9f3c2a`	`{"usuario":"000042"}`	Gravado com `--ttl 30m`
`config:moeda`	`BRL`	Configuração

15.3 Padrões

Criar só se não existir: `put usuario-email:ana@exemplo.com 000042 --se-ausente` garante e-mail único; o código 6 indica que já existe.

Contadores: `incr` e `decr` fazem o ciclo de ler, alterar e gravar com a versão, repetindo nos conflitos:

```
leme decr estoque:sku-1042           # um item vendido
leme incr pagina:inicio:visitas
```

Copyright (©) 2026 Erik Pimenta

Ler, alterar e gravar para outras mudanças, à mão:

```
V=$(leme get estoque:sku-1042 -o json | jq -r .versao)
Q=$(leme get estoque:sku-1042)
leme put estoque:sku-1042 $((Q - 1)) --se-versao "$V" || echo "repita: alguém alterou antes"
```

Listas paginadas: grave um item por elemento com prefixo comum (`fila:2026-10-23T19:00:01:000001`) e percorra com `scan fila: --limite 100 --depois <última>`.

Expiração em massa: prefira TTL a apagar depois; não gera lápides.

Mudar o nome de uma família de chaves: `rename` uma a uma para poucas chaves; para muitas, `export` do prefixo antigo e `import --prefixo` com o novo, conferindo antes com `--simular`.

15.4 O que evitar

EVITE	POR QUÊ	FAÇA ASSIM
Valores enormes que mudam pouco a pouco	Cada mudança regrava o valor inteiro	Divida em chaves menores
Uma única chave muito disputada	Todas as escritas vão para as mesmas três réplicas, e `incr` repete mais	Divida por partes (`contador:visitas:00` a `:15`) e some na leitura com `mget`
`scan` sem prefixo em produção	Consulta todos os nós	Use prefixos e `--limite`
Apagar e recriar a mesma chave em ciclo rápido	Acumula lápides	Sobrescreva ou use TTL

Copyright (©) 2026 Erik Pimenta

CAPÍTULO 16

Escalabilidade e operação

Fórmulas de capacidade, crescimento do cluster, backup e as métricas que mostram a saúde do banco.

16.1 Capacidade

RECURSO	FÓRMULA APROXIMADA POR NÓ
Chaves	N × total de chaves / número de nós
Memória do índice	chaves × (100 + tamanho médio da chave) bytes
Disco	chaves × (29 + chave + valor) bytes vivos, mais a fragmentação até a compactação
Rede por escrita	N cópias do registro saindo do coordenador

16.2 Crescendo

Adicione nós com `--entrar` (no Kubernetes, aumentando as réplicas do StatefulSet). O anel é recalculado e as chaves que mudaram de dono são enviadas automaticamente. Para reduzir, use `node leave`, um nó por vez, esperando `cluster nodes` mostrar o estado `saiu`.

16.3 Backup e restauração

O backup físico copia os segmentos de um nó:

01 Rode `leme node compact <nó>`, que fecha o segmento ativo.

02 Copie a pasta `dados/<id>` do nó; os segmentos fechados não mudam mais.

03 Para restaurar, pare o nó, devolva a pasta e suba de novo. As réplicas atualizam o que mudou depois do backup pelo reparo de leitura e pelas dicas.

16.4 Backup lógico

O backup lógico lê as chaves pelo cluster e grava um arquivo independente do formato do disco:

```
leme export --arquivo leme-2026-10-27.ndjson          # tudo
leme export pedido: --arquivo pedidos.ndjson          # um prefixo
leme import leme-2026-10-27.ndjson --simular          # confere sem gravar
leme import leme-2026-10-27.ndjson --se-ausente       # restaura só o que falta
```

Copyright (©) 2026 Erik Pimenta

CAMPO DO NDJSON	CONTEÚDO
`chave`	A chave, como texto
`valor`	O valor, como texto ou em base64
`codificacao`	`texto` ou `base64`
`ttl_ms`	Tempo restante em milissegundos, ausente sem expiração
`versao`	A versão no momento da exportação, só informativa

A exportação não é um instantâneo: cada chave é lida no momento em que a varredura passa por ela. Na importação, cada chave recebe uma versão nova do cluster de destino e o TTL é contado a partir dali. Use o backup físico para restaurar um nó inteiro e o lógico para restaurar chaves, migrar entre clusters ou guardar um arquivo legível.

16.5 Métricas do banco

MÉTRICA	SINAL DE ATENÇÃO
`leme_bytes_vivos / leme_bytes_disco`	Abaixo de 0,5: compacte
`leme_dicas_pendentes`	Crescendo: um nó está fora há tempo
`leme_reparos_leitura_total`	Subindo rápido: réplicas atrasadas, veja a rede
`leme_lapides`	Muito alto: apagar em massa; prefira TTL
Registros descartados em `node stats`	Fora do último segmento: problema de disco

As regras de alerta prontas para esses sinais estão em `deploy/prometheus/alertas.yml`, e o painel em `deploy/grafana/leme-dashboard.json`.

16.6 Diagnóstico

`leme doctor` reúne num só comando as checagens deste documento:

VERIFICAÇÃO	RELAÇÃO COM O BANCO
Membros, quórum e replicação	Se há réplicas vivas suficientes para W e R e nós suficientes para N (capítulo 12)
Equilíbrio do anel	Desvio da fatia de cada nó (capítulo 11)
Dicas pendentes	Escritas esperando réplicas fora do ar (capítulo 12)
Fragmentação	`1 - bytes vivos / bytes em disco` de cada nó (capítulo 10)
Latência p99 e erros	p99 de leitura e escrita e proporção de erros de cada nó
Versões	Todos os nós com a mesma versão do `lemed`

Cada verificação termina em `ok`, `aviso` ou `falha`, com uma sugestão; qualquer falha faz o comando sair com o código 8.

Copyright (©) 2026 Erik Pimenta